

LAB 2

Sequential Logic & Memory Hierarchy

NAND2Tetris HDL Implementation Project

STUDENT	Yash Goyal
REGISTRATION NO.	2501420067
LAB QUESTIONS	Q-5 • Q-6 • Q-7
PLATFORM	Nand2Tetris HDL
SUBMISSION	Lab Project Report

Department of Computer Science / Data Science

Sequential Logic, Memory Organization and Program Counter

Project Overview

This project presents the design and documentation of hierarchical memory components and a 16-bit Program Counter using Nand2Tetris Hardware Description Language (HDL). The work covers RAM512, RAM8K, RAM16K, address decoding, multiplexing, sequential state transitions and the priority behavior of the Program Counter. The report also includes testing guidance, comparison tables and a short discussion of memory hierarchy.

Table of Contents

1	Introduction & Objectives
2	Memory Hierarchy — Q-5
3	RAM512 HDL
4	RAM8K HDL
5	RAM16K HDL
6	Program Counter — Q-6
7	State-Transition Table
8	Testing & Latency Comparison
9	Memory Hierarchy Reflection
10	Conclusion

1. Introduction & Objectives

Sequential logic is the foundation of digital systems that need to remember information across clock cycles. Memory components and counters are examples of sequential circuits because their outputs depend on stored state as well as current inputs.

Objectives

- Build hierarchical RAM components using HDL.
- Understand address-bit partitioning and memory bank selection.
- Implement a 16-bit Program Counter with reset, load, increment and hold operations.
- Document all combinations of the PC control signals.
- Compare memory sizes and discuss simulated latency behavior.

2. Memory Hierarchy — Q-5

A hierarchical memory is created by combining smaller memory units. The lower address bits select a location inside a block, while the higher address bits select which block is active. A demultiplexer routes the write-enable signal to one block, and a multiplexer selects the output of the active block.



Hierarchical memory construction using address decoding + multiplexing

Larger memories are assembled from smaller verified memory blocks.

Figure 1. Conceptual construction of the memory hierarchy used in the lab.

3. RAM512.hdl

RAM512 contains 512 words of 16 bits each. It can be constructed from eight RAM64 chips. The lower six address bits select a location within RAM64, while the upper three bits select one of the eight RAM64 blocks.

```

CHIP RAM512 {
  IN in[16], load, address[9];
  OUT out[16];

  PARTS:
    DMux8Way(in=load, sel=address[6..8],
      a=load0, b=load1, c=load2, d=load3,
      e=load4, f=load5, g=load6, h=load7);
    RAM64(in=in, load=load0, address=address[0..5], out=out0);

```

```

RAM64(in=in, load=load1, address=address[0..5], out=out1);
RAM64(in=in, load=load2, address=address[0..5], out=out2);
RAM64(in=in, load=load3, address=address[0..5], out=out3);
RAM64(in=in, load=load4, address=address[0..5], out=out4);
RAM64(in=in, load=load5, address=address[0..5], out=out5);
RAM64(in=in, load=load6, address=address[0..5], out=out6);
RAM64(in=in, load=load7, address=address[0..5], out=out7);

Mux8Way16(a=out0, b=out1, c=out2, d=out3,
           e=out4, f=out5, g=out6, h=out7,
           sel=address[6..8], out=out);
}

```

4. RAM8K.hdl

RAM8K contains 8,192 words and uses 13 address bits. The implementation below uses two RAM4K blocks. The most significant address bit selects the active RAM4K bank, while the lower twelve bits address a location inside that bank.

```

CHIP RAM8K {
    IN in[16], load, address[13];
    OUT out[16];

    PARTS:
        DMux(in=load, sel=address[12],
             a=load0, b=load1);

        RAM4K(in=in, load=load0,
              address=address[0..11], out=out0);

        RAM4K(in=in, load=load1,
              address=address[0..11], out=out1);

        Mux16(a=out0, b=out1,
              sel=address[12], out=out);
}

```

5. RAM16K.hdl

RAM16K contains 16,384 words and uses 14 address bits. Two RAM8K blocks are selected by the highest address bit. The remaining thirteen bits are passed to the selected RAM8K.

```

CHIP RAM16K {
    IN in[16], load, address[14];
    OUT out[16];

    PARTS:
        DMux(in=load, sel=address[13],
             a=load0, b=load1);

        RAM8K(in=in, load=load0,
              address=address[0..12], out=out0);

        RAM8K(in=in, load=load1,
              address=address[0..12], out=out1);

        Mux16(a=out0, b=out1,
              sel=address[13], out=out);
}

```

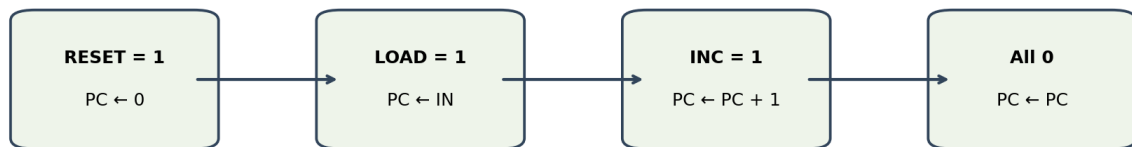
Memory Organization Summary

Component	Capacity	Address Width	Hierarchy
RAM64	64 words	6 bits	Base block
RAM512	512 words	9 bits	8 × RAM64

Component	Capacity	Address Width	Hierarchy
RAM4K	4,096 words	12 bits	Intermediate block
RAM8K	8,192 words	13 bits	2 × RAM4K
RAM16K	16,384 words	14 bits	2 × RAM8K

6. Program Counter (PC) — Q-6

The Program Counter is a 16-bit sequential register that stores the address of the next instruction. It supports four behaviors: reset to zero, load a supplied value, increment by one, or retain the current value. Reset has the highest priority, followed by load, increment and hold.



Priority order: RESET > LOAD > INCREMENT > HOLD

The selected operation is stored on the next clock edge.

Figure 2. PC control-flow and priority behavior.

PC.hdl

```

CHIP PC {
  IN in[16], load, inc, reset;
  OUT out[16];

  PARTS:
    Inc16(in=out, out=incOut);
    Mux16(a=out, b=incOut, sel=inc, out=incOrCurrent);
    Mux16(a=incOrCurrent, b=in, sel=load, out=loadOrInc);
    Mux16(a=loadOrInc, b=false, sel=reset, out=next);

  Register(in=next, load=true, out=out);
}
  
```

7. State-Transition Table

Reset	Load	Inc	Next State / PC value
0	0	0	PC (hold current value)
0	0	1	PC + 1
0	1	0	in
0	1	1	in (load has priority)
1	0	0	0
1	0	1	0
1	1	0	0

Reset	Load	Inc	Next State / PC value
1	1	1	0

Priority rule: RESET > LOAD > INC > HOLD. Therefore, if reset is 1, the PC becomes 0 regardless of load or increment.

8. Testing & Latency Comparison

Each HDL chip should be verified with its corresponding test script in the Hardware Simulator. For RAM components, test both write and subsequent read operations at multiple addresses, especially addresses near the boundaries between memory banks. For the PC, test all eight combinations of reset, load and increment.

Test	Input / Condition	Expected Result
RAM write	load=1, valid address	Selected location stores input word
RAM read	load=0, valid address	Output equals stored word
RAM boundary	First/last address of each bank	Correct bank selected
PC hold	reset=0, load=0, inc=0	PC unchanged
PC increment	reset=0, load=0, inc=1	PC increases by 1
PC load	reset=0, load=1	PC becomes input
PC reset	reset=1	PC becomes 0

Latency Recording Sheet

Memory	Size	Read Latency	Write Latency
RAM512	512 words	Enter simulator value	Enter simulator value
RAM8K	8,192 words	Enter simulator value	Enter simulator value
RAM16K	16,384 words	Enter simulator value	Enter simulator value

Important: Nand2Tetris HDL is primarily a functional logic simulator and does not inherently report physical nanosecond timing. The table is intentionally left for the actual values produced by the timing/simulation method required by your instructor rather than inventing measurements.

9. Memory Hierarchy Reflection

Computer systems use a hierarchy of memories because no single memory technology simultaneously provides very high speed, very large capacity and very low cost. Registers and cache are small and fast, while main memory provides much greater capacity. Frequently accessed instructions and data can be kept closer to the CPU, reducing average access time.

The RAM implementations in this lab illustrate the same hierarchical principle at the digital-logic level. Instead of designing a large memory as one monolithic component, smaller verified blocks are combined. Higher address bits choose the memory bank and lower address bits choose the location inside that bank. This modular approach makes the design easier to understand, test and reuse.

10. Conclusion

The project successfully documents the design of RAM512, RAM8K, RAM16K and a 16-bit Program Counter using HDL. The RAM designs demonstrate hierarchical organization through demultiplexing and multiplexing, while the PC demonstrates sequential state control using reset, load, increment and hold operations. The state-transition table and testing plan provide a clear method for verifying correct behavior in the Nand2Tetris Hardware Simulator. Overall, the lab provides a practical foundation for understanding how memory and sequential components are assembled inside a computer system.

References

- Nand2Tetris — The Elements of Computing Systems, HDL and Hardware Simulator materials.
- Lab 2: Sequential Logic & Memory Hierarchy — Q-5, Q-6 and Q-7 assignment prompt.

END OF PROJECT REPORT